ARCHITECTURE GUIDE · TECHNICAL REFERENCE

THE TRACKER

Architecture Guide

Stack: Next.js 16.2 App Router · React 19 · TypeScript 5 · Supabase PostgreSQL · Tailwind 4

Auth: Google OAuth 2.0 · Supabase Auth · JWT · HTTP-only Cookies

Database: 15 Tables · 11 Migrations · RLS on All · 6 Triggers

Deployment: Vercel Edge · Supabase Cloud · No Custom API Routes

---

Hussein A-H · github.com/HusseinA-H/THE-TRACKER · June 2026

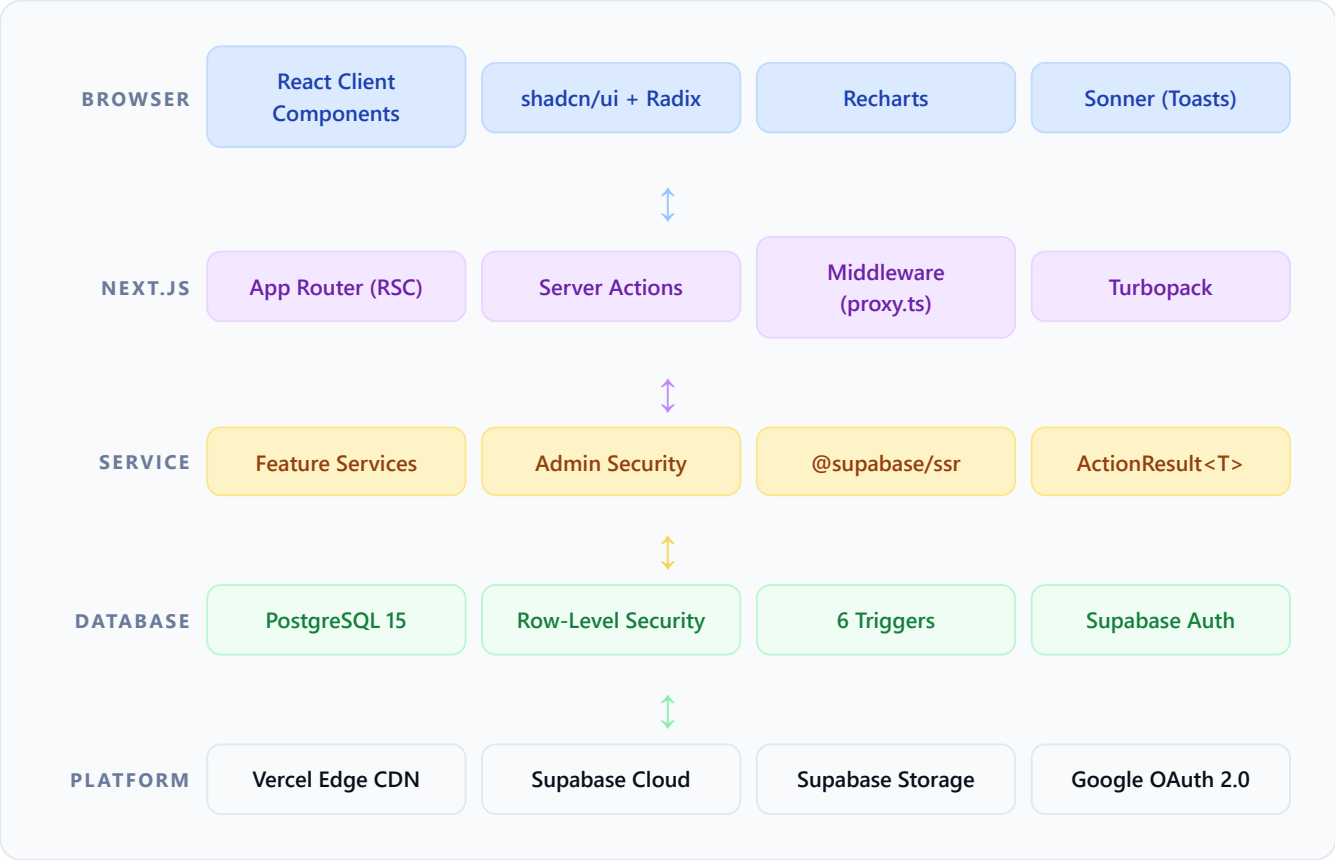
ARCHITECTURE GUIDE

System Architecture Overview

THE TRACKER is a monolithic Next.js application with zero custom API routes. All server-side mutations run through Next.js Server Actions. Data access is enforced at the PostgreSQL layer via Row-Level Security, creating a true defense-in-depth architecture.

24	0	15	6	11	3
Routes	API Routes	DB Tables	Triggers	Migrations	Auth Roles

Layered Architecture



Request Lifecycle — Full Read Flow

```
GET /dashboard (Server Component page render)
```

```
// 1. Browser hits Vercel Edge CDN GET /dashboard → Vercel routes to serverless function // 2. proxy.ts middleware runs FIRST (before any page) export async function middleware(request) { const supabase = createServerClient(...); const { data: { user } } = await supabase.auth.getUser(); // Writes refreshed session cookies back to response } // 3. DashboardLayout (Server Component) const { data: { user } } = await supabase.auth.getUser(); if (!user) redirect("/login"); const { data: profileRow } = await supabase.from("profiles").select("*")... // 4. Page Component (Server Component) const stats = await getDashboardStats(); // Service layer call // 5. getDashboardStats() calls Supabase // PostgreSQL executes query → RLS filter applied → data returned // RLS: auth.uid() = user_id automatically filters rows // 6. RSC renders HTML server-side → streamed to browser // Client components hydrate with React 19
```

Request Lifecycle — Mutation Flow (Server Action)

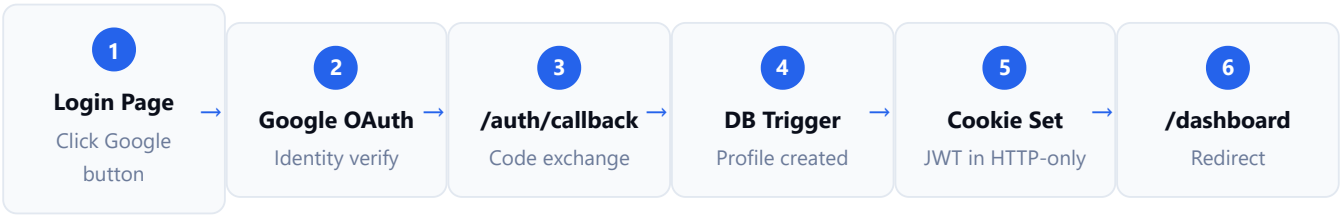
POST via Server Action — logSetAction()

```
// 1. Client component calls action const result = await addEntryAction({ workoutId, exerciseId, weight, reps }); // 2. Server Action ("use server") runs on server export async function addEntryAction(input: LogSetInput) { const entry = await createWorkoutEntry(input); // service revalidatePath("/dashboard/workouts/active"); // invalidate cache return { success: true, data: entry }; } // 3. createWorkoutEntry() in workout-service.ts const supabase = await createClient(); // auth-aware client const { data: { user } } = await supabase.auth.getUser(); if (!user) throw new Error("Unauthorized"); // 4. Insert to PostgreSQL // RLS CHECK POLICY validates workout belongs to auth.uid() await supabase.from("workout_entries").insert(entryRow); // 5. PR check: compare against personal_records // If new PR: upsert personal_records + insert pr_history row // 6. revalidatePath() triggers Next.js to re-fetch RSC data // Active workout page re-renders with new set visible
```

AUTH ARCHITECTURE

Authentication & Authorization Architecture

Google OAuth 2.0 Flow



Middleware Architecture (proxy.ts)

src/proxy.ts — Next.js Middleware

```
export async function middleware(request: NextRequest) { let supabaseResponse = NextResponse.next({ request }); const supabase = createServerClient( process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!, { cookies: { getAll() { return request.cookies.getAll() }, setAll(cookiesToSet) { // CRITICAL: write refreshed cookies back to response cookiesToSet.forEach(({ name, value, options }) => supabaseResponse.cookies.set(name, value, options) ); } } } ); // This call triggers token refresh if needed const { data: { user } } = await supabase.auth.getUser(); return supabaseResponse; } export const config = { matcher: ['/((?!_next/static|favicon).*)'] };
```

RBAC Architecture

ROLE	DATABASE VALUE	APPLICATION ACCESS	PROTECTED BY
user	'user'	All /dashboard/* routes + own data only	RLS policies (auth.uid() = user_id)
admin	'admin'	/administration/* (except users/settings)	verifyAdmin() + RLS
super_admin	'super_admin'	All routes including user management	verifySuperAdmin() + DB trigger

src/features/admin/security.ts

```
export async function verifyAdmin() { const { user } = await getAuthenticatedUser(); const profile = await supabase.from("profiles").select("role").eq("id", user.id); if (!['admin','super_admin'].includes(profile.role)) throw new Error("Forbidden"); return { user, role: profile.role }; } // Every admin server action starts with: const { role } = await verifyAdmin();
```

DATABASE ARCHITECTURE

Database Schema Architecture

Entity Relationship Map

```
auth.users (Supabase managed) │ └─[1:1 CASCADE]─► profiles ├─[1:N CASCADE]─► workouts │ └─[1:N CASCADE]─► workout_entries ◄─[1:N RESTRICT]─ exercises │ └─[N:1 SET NULL]─► personal_records │ ├─[1:N CASCADE]─► weight_logs ◄──────────────────────────── TRIGGER SYNC │ │ ├─[1:N CASCADE]─► body_measurements ◄──────────────────────────────┘ ├─[1:N CASCADE]─► progress_photos ├─[1:N CASCADE]─► personal_records │ └─[1:N CASCADE]─► personal_records_history └─[1:1 CASCADE]─► user_schedule_settings exercises │ ├─[1:N CASCADE]─► exercise_alternatives (exercise_id + alternative_id → exercises) ├─[1:N CASCADE]─► workout_template_exercises ◄── workout_templates │ └─ workout_packages (via workout_package_templates) └─[1:N CASCADE]─► personal_records_history
```

Complete Table Schemas

profiles

PK id	uuid
email	text UNIQUE NOT NULL
display_name	text
avatar_url	text
role	'user'|'admin'|'super_admin'
created_at	timestamptz DEFAULT NOW()
updated_at	timestamptz DEFAULT NOW()

exercises

PK id	uuid gen_random_uuid()
FK user_id	uuid NULLABLE → profiles
name	text NOT NULL
primary_muscle_group	text (CHECK)
secondary_muscle_group	text
category	text (Upper|Lower|Cardio)
video_url	text
is_custom	boolean DEFAULT true

💪 workouts

PK	id	uuid
FK	user_id	uuid NOT NULL → profiles
FK	template_id	uuid NULLABLE → templates
	name	text DEFAULT 'Workout'
	notes	text
	started_at	timestamptz DEFAULT NOW()
	completed_at	timestamptz NULLABLE

📝 workout_entries

PK	id	uuid
FK	workout_id	uuid NOT NULL CASCADE
FK	exercise_id	uuid NOT NULL RESTRICT
	set_number	integer NOT NULL
	weight	numeric(6,2) DEFAULT 0
	reps	integer DEFAULT 0
	set_type	warmup|working|top|failure
	is_pr	boolean DEFAULT false
	estimated_1rm	numeric(6,2)

🏆 personal_records

PK	id	uuid
FK	user_id	uuid NOT NULL
FK	exercise_id	uuid NOT NULL
	max_weight	numeric(6,2) DEFAULT 0
	max_estimated_1rm	numeric(6,2) DEFAULT 0
	max_volume	numeric(8,2) DEFAULT 0
FK	set_id	uuid → workout_entries

UNIQUE (user_id, exercise_id)

📋 workout_templates

PK	id	uuid
FK	user_id	uuid NULLABLE → profiles
	name	text NOT NULL
	description	text
	estimated_duration	integer (minutes)
	muscle_focus	text

user_id IS NULL = system template

Database Triggers

TRIGGER NAME	EVENT	FUNCTION	PURPOSE
on_auth_user_created	AFTER INSERT on auth.users	handle_new_user()	Auto-create profile on first Google login
profiles_updated_at	BEFORE UPDATE on profiles	update_updated_at()	Auto-timestamp updated_at
personal_records_updated_at	BEFORE UPDATE on personal_records	update_updated_at()	Auto-timestamp updated_at

TRIGGER NAME	EVENT	FUNCTION	PURPOSE
protect_profile_role	BEFORE UPDATE on profiles	check_role_update()	Prevent unauthorized role escalation
sync_template_exercise_order	BEFORE INSERT/UPDATE on template_exercises	sync_template_exercise_order()	Keep sequence_number and exercise_order in sync
trigger_sync_weight_to_measurements	AFTER INSERT/UPDATE on weight_logs	sync_weight_to_measurements()	Mirror weight log to body_measurements
trigger_sync_measurements_to_weight	AFTER INSERT/UPDATE on body_measurements	sync_measurements_to_weight()	Mirror measurement weight to weight_logs

Row-Level Security Summary

```
-- Pattern 1: User-scoped data (most tables) USING (auth.uid() = user_id) WITH CHECK (auth.uid() =
user_id) -- Pattern 2: System + user data (exercises, templates) USING (user_id IS NULL OR auth.uid() =
user_id) -- Pattern 3: Indirect ownership (workout_entries) USING ( EXISTS ( SELECT 1 FROM workouts
WHERE workouts.id = workout_entries.workout_id AND workouts.user_id = auth.uid() ) )
```

MIGRATION STRATEGY

Migration Architecture

00001 — Foundation

create_tables.sql

6 core tables: profiles, exercises, workouts, workout_entries, weight_logs, personal_records. All foreign keys, check constraints. NUMERIC precision for weights.

00002 — Security

rls_policies.sql

ALTER TABLE ... ENABLE ROW LEVEL SECURITY on all 6 tables. Defines SELECT/INSERT/UPDATE/DELETE policies. Indirect ownership pattern for workout_entries.

00003 — Automation

triggers.sql

handle_new_user() trigger for auto profile creation on OAuth. update_updated_at() for profiles and personal_records. SECURITY DEFINER execution context.

00004 — Performance

indexes.sql

B-tree indexes on all user_id columns, log_date, completed_at. Covers all common query filter patterns with O(log n) performance.

00005 — RBAC

add_role_to_profiles.sql

ADD COLUMN role TEXT DEFAULT 'user'. CHECK constraint: user|admin|super_admin. protect_profile_role trigger prevents unauthorized role escalation via UPDATE. Only super_admin can change roles.

00006 — Exercise RLS

update_exercise_rls_policies.sql

Refines exercise visibility: user_id IS NULL means system exercises (visible to all authenticated users). user_id set means user-private custom exercise. Enables admin to create system exercises.

00007 — Fields

add_exercise_fields.sql

ADD COLUMN category, equipment, difficulty, alternative_exercise, notes to exercises. Enables categorization (Upper/Lower/Cardio) and coaching notes.

00008 — Templates

workout_templates.sql

Creates workout_templates and workout_template_exercises tables. Adds template_id FK to workouts. Adds set_type to workout_entries. Seeds 6 system templates with all exercises and rep targets. RLS policies: system templates visible to all.

00009 — Packages

refine_workout_system.sql

Creates workout_packages and workout_package_templates junction tables. Seeds "Upper & Lower Split" package containing all 6 templates ordered by sequence.

00010 — Athlete Upgrade

athlete_upgrade_system.sql

exercise_alternatives (self-join, 3 ordered swaps, self-reference constraint). personal_records_history (timestamped PR events). body_measurements (8 measurements + date). progress_photos (URL + front/side/back). user_schedule_settings. Bi-directional weight sync triggers. Extends personal_records with max_volume column.

00011 — Storage

progress_photos_bucket.sql

Configures Supabase Storage bucket "progress-photos". RLS policies: authenticated INSERT (own folder), authenticated SELECT/DELETE (own files only). MIME type restrictions.

FRONTEND ARCHITECTURE

Frontend Module Architecture

Feature-Based Module Pattern

```
src/features/ ├── admin/ │ ├── components/ # AdminDashboard, UserTable, etc. │ ├── services/ │ │ └── admin-service.ts # getSystemStats, getAllUsers │ ├── actions.ts # Server Actions for admin mutations │ ├── security.ts # verifyAdmin(), verifySuperAdmin() │ └── types.ts # AdminStats, SystemUser types │ ├── exercises/ │ ├── components/ # ExerciseTable, ExerciseDetailDrawer │ ├── services/ │ │ └── exercise-service.ts │ ├── actions.ts │ └── types.ts │ ├── workouts/ │ ├── components/ # WorkoutCard, ActiveWorkout, SetInputRow │ ├── services/ │ │ ├── workout-service.ts # 1105 lines — core workout logic │ │ └── schedule-service.ts # Cycle, streak, calendar │ ├── utils/ │ │ └── cycle.ts # Pure: CYCLE_DAYS, calculateCycleDay() │ ├── actions.ts │ ├── schemas.ts # Zod validation schemas │ └── types.ts │ ├── weight/ │ ├── components/ # WeightChart, MeasurementsForm, PhotosGrid │ ├── services/ │ │ ├── weight-service.ts │ │ ├── measurements-service.ts │ │ └── photos-service.ts │ ├── actions.ts │ └── types.ts │ ├── auth/ │ ├── components/ # LoginButton, AuthCard │ └── actions.ts # signInWithGoogle, signOut │ ├── dashboard/ │ ├── components/ # StatsGrid, WeeklyHeatmap │ ├── services/ │ │ └── dashboard-service.ts # getDashboardStats() — 5 queries │ └── types.ts │ ├── profile/ │ ├── services/ │ │ └── profile-service.ts # getProfile, mapProfileRow │ └── types.ts │ └── records/ ├── components/ # RecordsTable, PRHistoryDrawer └── services/ └── records-service.ts
```

Server vs Client Component Decisions

COMPONENT	TYPE	REASON
DashboardLayout	Server	Needs auth.getUser() + profile DB query before render
DashboardPage	Server	Fetches stats server-side via getDashboardStats()
ExercisesPage	Server (shell) + "use client" (table)	Initial data from server; filtering is client-side
ActiveWorkout	Client ("use client")	useState, useEffect for timer, event handlers for set inputs
WeightChart	Client	Recharts requires DOM access — cannot render on server

COMPONENT	TYPE	REASON
CalendarView	Client	date-fns cycle calculation uses current date (client-aware)
ExerciseDetailDrawer	Client	Sheet/Dialog state, tab switching, optimistic updates
AuthProvider	Client	React Context must be a client component
Sidebar	Client	usePathname() for active state; conditional Admin link from useAuth()

Data Fetching Patterns

🔵 Server Component Pattern

Used for initial page data. Service functions called directly inside Server Components. No useEffect, no loading states needed. Data is fresh on every request (or cached via Next.js cache).

```
async function ExercisesPage() { const exercises = await getExercises(); return <ExerciseTable exercises={exercises} />; }
```

⚡ Server Action Pattern

Used for all mutations. Client component calls action, awaits ActionResult<T>, shows toast on success/error. revalidatePath() triggers Next.js to re-fetch affected RSC data.

```
const res = await addEntryAction(input); if (res.success) toast.success("Set logged!"); else toast.error(res.error);
```

SECURITY

Security Architecture — Defense in Depth

1. **Vercel Edge Middleware (proxy.ts)**
   Every HTTP request hits middleware first. Validates JWT, refreshes if expired. Cannot be bypassed. Handles all routes matching /((?!_next/static|favicon).*)

2. **Next.js Layout Auth Check**
   DashboardLayout and AdministrationLayout independently call supabase.auth.getUser() server-side. redirect() on failure. Belt-and-suspenders with middleware.

3. **Server Action User Validation**
   Every server action that touches user data calls supabase.auth.getUser() at the start. Throws "Unauthorized" if no session. Prevents Server Action abuse.

4. **Admin Role Verification (verifyAdmin)**
   All admin actions and admin page layouts call verifyAdmin() which reads the profile.role from database (not JWT claims). Cannot be faked client-side. Throws 403 Forbidden on insufficient role.

5. **PostgreSQL Row-Level Security**
   RLS enabled on all 15 tables. auth.uid() evaluated by PostgreSQL at query time. Even if application code has a bug, users cannot read or modify other users' data. Database-enforced, cannot be disabled by application code.

6. **Database Trigger: Role Protection**
   protect_profile_role trigger runs BEFORE UPDATE on profiles. If role is being changed and the requesting user is not super_admin, silently reverts the change. Prevents SQL injection or privilege escalation even with direct DB access via the anon key.

Environment Variables

VARIABLE	ACCESS	USED FOR
`NEXT_PUBLIC_SUPABASE_URL`	Public (browser + server)	Supabase project endpoint
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Public (anon JWT)	Client-side Supabase auth — RLS enforces data isolation
`SUPABASE_SERVICE_ROLE_KEY`	Server-only (secret)	Admin operations that bypass RLS (not used in current version)

⚠️ **Anon Key Safety**

The anon key is intentionally public. It grants Supabase Auth access and is constrained by RLS policies. Without a valid authenticated user session, all data queries return empty. The security model relies on PostgreSQL, not key secrecy.

PERFORMANCE

Performance Architecture

OPTIMIZATION	MECHANISM	ESTIMATED IMPACT
React Server Components	Zero client JS for data fetching; HTML rendered server-side	50–70% smaller client bundle
Turbopack	Next.js 16 dev server — Rust-based bundler	~10x faster HMR than webpack
No hydration waterfalls	RSC streams HTML; client components hydrate independently	Faster Time-to-Interactive
Database indexes	B-tree on user_id, log_date, exercise_id, template_id, completed_at	O(log n) query on all common filters
Selective revalidation	revalidatePath() targets specific route segments only	Minimal RSC cache invalidation
Supabase PgBouncer	Connection pooling for serverless — prevents connection exhaustion	Handles 100s of concurrent requests
Vercel CDN	Static assets (CSS, fonts, images) served from Edge locations	<50ms asset delivery globally
Zero state management runtime	No Redux/Zustand bundle overhead	~15–30KB smaller bundle

DEVELOPER GUIDE

Developer Reference

Key Invariants — Never Break These

- **Every new admin action MUST call verifyAdmin()** — never rely on frontend role checks alone
- **Every new database table MUST have RLS enabled** — enable in the same migration that creates the table

- **cycle.ts must remain import-free** — no server imports, no next/headers, no supabase — used by client components
- **All mutations use Server Actions** — never create /api/* route handlers for user data mutations
- **Migration files must be sequentially numbered** — audit existing files before naming a new migration
- **revalidatePath() must be called after every mutation** — otherwise RSC data will be stale
- **TypeScript errors are breaking** — `npm run build` must pass before any commit

Adding a New Feature Checklist

#	STEP	LOCATION
1	Create feature directory	`src/features/<feature-name>/`
2	Define TypeScript types	`types.ts` in feature dir
3	Write service layer	`services/<feature>-service.ts`
4	Write server actions	`actions.ts` ("use server" at top)
5	Build components	`components/` in feature dir
6	Create page route	`src/app/dashboard/<feature>/page.tsx`
7	Add route constant	`src/config/routes.ts`
8	Add nav item (if needed)	`navItems` in routes.ts + sidebar.tsx + bottom-nav.tsx
9	Create migration	`supabase/migrations/000XX_<feature>.sql`
10	Enable RLS in migration	ALTER TABLE ... ENABLE ROW LEVEL SECURITY; + policies
11	Run migration	`npx supabase db push`
12	Verify build	`npm run build`

THE TRACKER — Architecture Guide

Hussein A-H · github.com/HusseinA-H/THE-TRACKER · 2026